SWITCH

COMPLETE MERGED MASTER SPECIFICATION

Original Product, UX & USP Master Document + Alexa+ Integration and Implementation Specification

Purpose: one coordinated reference for Person 1, Person 2 and Person 3. The original 22-page product document is included intact, followed by the 28-page Alexa+ extension. The two source documents are not silently rewritten; this short guide resolves how to read and implement them together.

Document order and authority

PART	PAGES IN THIS FILE	HOW TO USE IT
Reader guide	1–2	Shared decisions, ownership and integration guardrails.
Part I — Original baseline	3–24 (original source pages 1–22)	Product purpose, USP, UX, workflow, schemas, trust boundaries, existing team workstreams and demo. These 22 pages are retained unchanged.
Part II — Alexa+ extension	25–52 (extension source pages 1–28)	Additional architecture, flowcharts, wireframes, adapter contracts, Person 1/2/3 split, build gates and tests.

Canonical product flow



Alexa+ is an interaction/adapter layer — it must not bypass confirmation or the deterministic verifier.

If the Alexa+ route is simulated, label it SIMULATED; do not imply live account/device access.

Alexa+ can provide a conversational entry point or call supported SWITCH tools. It does not replace the core correction → confirmation → versioned requirement → deterministic replay → evidence → human review workflow.

Shared implementation decisions — read before splitting work

Preserve the original USP: a household correction explicitly confirmed by a person becomes a contextual, versioned regression requirement, which is replayed when a related routine changes. Alexa+ is an added interaction/integration layer—not a replacement for that core.

One Alexa+ route, not three competing implementations. Person 3 must verify the current hackathon/developer requirements and access, record the decision, and implement the primary route. The proposed choices are a supported MCP server, an Agent Skill, or a clearly labelled in-app simulated Alexa+ experience if live access is unavailable/unsupported. Do not present a simulation as live integration.

Existing prototype status remains the baseline: the original document says the prototype simulates devices, routines, saved lessons and test results, and is not connected to physical devices, an Amazon account or live Alexa services. Treat any live connection as unimplemented until tested evidence proves otherwise.

Accountable work division

OWNER	ACCOUNTABLE FOR	HANDOFF / ACCEPTANCE
PERSON 1 — UX / Frontend	App shell and original pages; interactive house/room/device UI; integration status page; accessible state-driven UI; replay timeline, diff and reports presentation.	UI consumes shared API/contracts; room/device state matches trace; visible SIMULATED / CONNECTED status; no hardcoded pass/fail results; responsive/accessibility checks.
PERSON 2 — Deterministic core / Backend	Canonical routine/requirement/replay schemas; versioning; deterministic simulator and event order; assertion evaluator; APIs and persistence if feasible; trace/report generation.	Same inputs produce same trace/result; unknown states are INCONCLUSIVE; repair must rerun relevant suite; APIs and fixtures documented; test evidence supplied to Persons 1 and 3.
PERSON 3 — AI / Evaluation / Alexa+ Integration	Correction interpretation and clarification; schema-validated constrained repair proposals; benchmark scenarios; Alexa+ route selection and adapter/tool/skill; end-to-end integration and fallback.	Route decision recorded; tool schemas and error behavior shared; adapter calls Person 2 APIs; no bypass of confirmation/verifier; demo explicitly identifies live vs simulated behavior.

Non-negotiable integration rules

1. Freeze one shared contract before parallel implementation: IDs, room/device IDs, routine and requirement schemas, result enums, trace event shape, version rules, provenance and error format.

2. Only the deterministic evaluator assigns PASS / FAIL / INCONCLUSIVE. AI may interpret, clarify, explain or suggest; it does not decide the result.

3. A correction stays DRAFT until the user explicitly confirms it. A candidate repair is not deployed automatically; it must be reviewed and regression-tested.

4. Use the same canonical state/trace data for the house view, timeline, reports and Alexa+ responses. Never let separate UI/demo values disagree.

5. If an operation or device is unsupported, access is unavailable, or required evidence is missing, return a visible limitation/error or INCONCLUSIVE—not a fabricated success.

6. Keep the original document’s requirements and all working existing behavior. If code and documents differ, inspect the actual code, record the discrepancy and agree on a deliberate update.

Final status rule: every major feature must be labelled IMPLEMENTED, SIMULATED, BLOCKED or NOT IMPLEMENTED, with evidence. This PDF is an implementation specification; it does not prove that live Alexa+ integration exists.

SWITCH

UPDATED PRODUCT, UX & USP MASTER DOCUMENT

Smart-home routine verification, replay and regression requirements

Document purpose	**Updated single source of truth for product scope, visual direction, USP, architecture and demo.**
Key UX update	Premium interactive house interface, larger typography, meaningful state-driven motion.
Key USP update	Explain the correction → confirmed requirement → future replay workflow concretely and honestly.
Status boundary	Prototype behavior is simulated unless a real capability is implemented and tested.

Updated 22 October 2026 • Project blueprint, not a claim that every described capability is already implemented

Document map

1. 1. Executive summary
2. 2. Product problem and user need
3. 3. USP — what is actually different?
4. 4. UX direction and visual design brief
5. 5. Page-by-page product experience
6. 6. End-to-end functional workflow
7. 7. System architecture and trust boundaries
8. 8. Requirement model and API contract
9. 9. AI/ML strategy
10. 10. Data and evaluation plan
11. 11. Security, privacy and safety boundaries
12. 12. Technology and implementation sequencing
13. 13. Team responsibilities and integration
14. 14. Demo script and judging narrative
15. 15. Risks, open questions and definition of done
16. 16. Research basis and links

1. Executive summary

SWITCH is a proposed smart-home routine verification layer. It helps a household preserve explicitly confirmed corrections as reusable, versioned regression requirements, then replay those requirements when a related automation changes.

One-sentence product pitch

> **A fix today should not become a failure tomorrow. SWITCH turns a household's confirmed smart-home corrections into repeatable checks, then shows whether a proposed routine change still satisfies them before a person approves it.**

The core product is not just a smart-home dashboard, a generic chatbot, or a one-time conflict detector. The intended workflow links a household correction to a structured requirement, stores that requirement as a versioned test, and reruns it against future routine edits.

Current implementation status — be explicit

- An interactive browser prototype exists for the correction-to-regression-test concept.
- The current prototype simulates devices, routine behavior, saved lessons, test results and suggested fixes.
- The prototype is not connected to physical devices, an Amazon account, or live Alexa services; it has no configured backend, database, or live LLM.
- Any on-screen metrics, test outcomes, device states and fixes in the prototype are demonstration data, not measured performance.

This document updates the project blueprint with a stronger product interface, a clearer second-page USP explanation, and a concrete demo story. It preserves the research boundary: the differentiation is a hypothesis to validate, not a proven world-first claim.

2. Product problem and user need

The problem

Smart-home routines are often created independently, but they can interact through shared devices, occupancy, time, modes and event ordering. A routine may appear correct by itself and still cause a household outcome the user explicitly asked to avoid. After a person fixes that outcome, a later edit can accidentally reintroduce it.

Illustrative scenario

17. At night, a Goodnight routine turns off lights.
18. An occupancy condition is accidentally removed from the hallway-light action.
19. Someone is still in the hallway, but the changed routine turns the light off.
20. The household corrects the routine. Weeks later, another edit reintroduces the same failure.
21. SWITCH's intended value is to preserve the confirmed correction as a test and replay it when the routine changes again.

Why the correction must be confirmed

A single manual override does not always express a permanent rule. A person may turn a light back on because of a temporary situation, a guest, a sensor issue, or a changed preference. SWITCH should capture context, propose a structured interpretation, and ask the user to confirm whether it should become a durable requirement. Unconfirmed observations must not silently become household policy.

Product outcome

Before a proposed routine change is approved, the user can inspect the relevant saved requirements, replay result, event trace, failed assertion and proposed change. The product helps the user make an informed decision; it does not autonomously guarantee safety or deploy changes without approval.

3. USP — what is actually different?

A fix today should not become a failure tomorrow.

The differentiating hypothesis

SWITCH's proposed differentiator is a consumer-facing correction-to-regression workflow: an explicitly confirmed household correction becomes a contextual, reviewable, versioned requirement that can be replayed against later routine edits.

What that means in plain English

Most people do not want to write test scripts for their home. They notice a bad outcome and correct it. SWITCH is designed to let them explain the correction in ordinary language, confirm exactly what should remain true, and reuse that requirement as a test in the future. When a routine changes, the system does not only ask whether the new routine works in one isolated example; it also checks whether the household's previously confirmed expectation still holds in the scenarios represented by the test suite.

Concrete walkthrough

22. CORRECT — The user says: "Don't turn off the hallway light while someone is there."
23. CONFIRM — SWITCH proposes a structured requirement: if hallway occupancy is true, a Goodnight routine must not set the hallway light to off. The user confirms the scope and conditions.
24. SAVE — The requirement is stored with an ID, version, source correction, conditions, expected behavior and status. It becomes part of the household's regression suite.
25. CHANGE — A later edit removes the occupancy condition from the Goodnight routine.
26. REPLAY — SWITCH executes a deterministic modeled scenario with occupancy=true and records the routine's actions and state transitions.

27. REPORT — The requirement fails. The UI points to the action that turned off the light and shows the violated assertion.
28. REPAIR AND RECHECK — SWITCH may suggest restoring the occupancy guard. The proposed change is not accepted as correct merely because an AI suggested it; the deterministic suite must be rerun.
29. APPROVE — The person reviews the evidence and chooses whether to accept the change. A passing modeled test is evidence about the model, not a guarantee about the physical home.

How to explain the difference without overclaiming

Generic capability / claim	SWITCH's proposed focus
Detect a conflict once	Preserve a confirmed correction as a reusable requirement.
Show a suggested fix	Show which saved expectation failed, the event trace, the proposed change and the rerun result.
Use AI to explain or generate a change	Use AI only for interpretation or suggestions; deterministic checks own pass/fail.
Test a current scenario	Replay relevant saved requirements after a related routine edit.
Promise that a home is safe	Report exactly which modeled requirements passed, failed or could not be evaluated.

What we must not claim

- Do not claim "world's first," "guaranteed safe," or "prevents all conflicts."
- Do not claim that no existing product implements this workflow; the preliminary audit is not exhaustive.
- Do not present natural-language testing, conflict detection, formal verification, repair suggestions, or regression testing alone as novel.
- Do not claim a live Alexa+ integration unless it has been implemented and tested with supported access.
- Do not say that an AI-generated fix is verified until the deterministic verifier has rerun the relevant requirements.

How to validate the USP

- Compare the correction capture, confirmation, persistence and cross-edit replay workflow with Alexa Automated Testing Service, TAPFixer, SHACR, Hirz, and relevant Home Assistant / SmartThings / openHAB / Node-RED workflows.
- Interview 5–10 smart-home users about repeated routine failures, current workarounds, and whether preserving a correction as a test is valuable.
- Measure how often users confirm or edit a proposed requirement, whether saved requirements catch deliberately reintroduced regressions, and whether the event trace helps users understand the result.
- Phrase the claim as a product hypothesis until competitor review and user validation support a stronger conclusion.

4. UX direction and visual design brief

The interface should feel like a premium, interactive smart-home control product with an engineering verification layer—not a generic admin dashboard. Use the supplied reference image as the visual bar for depth, spatial layout and live device feedback. Keep the interface legible at a glance and prioritize meaningful interaction over decoration.

Visual language

- Palette: graphite and deep slate surfaces, restrained blue/cyan highlights, warm neutral text, and semantic status colors. Avoid pure-black walls of UI; use layered slate surfaces and clear contrast.
- Typography: increase the base size and hierarchy. Suggested desktop body 16–18 px, key labels 14–16 px, section headings 22–28 px, primary page title 34–44 px, and hero statement 40–56 px where space permits. Do not shrink essential information into tiny chips.
- Layout: generous spacing, clear grouping, strong alignment, consistent card radii and subtle borders. Avoid a cluttered grid of same-weight cards.
- Motion: use short, purposeful transitions for replay steps, device state changes, timeline events and pass/fail states. Respect reduced-motion preferences.

- Accessibility: keyboard navigation, visible focus, adequate contrast, semantic controls, labels for icon-only buttons, and non-color status indicators.
- Responsive behavior: desktop-first spatial view can collapse to a simplified house overview plus stacked panels on narrower screens.

Primary application shell

30. Left navigation: Home Overview, Routines, Replay Lab, Saved Requirements, Reports, Devices / Integrations, Settings. Keep the active section unmistakable.
31. Top bar: household selector, environment badge (SIMULATED / CONNECTED), search or command entry, notifications and user/profile controls.
32. Main canvas: interactive house cutaway or room map. Each room and device should be selectable; state overlays must reflect the same simulated state as the timeline and side panels.
33. Right rail: selected room/device controls, active routines, current alerts, and a compact explanation of the latest verification state.
34. Bottom action strip: Run Replay, Compare Changes, View Trace, and Review Proposed Fix. Actions should be context-aware and disabled when prerequisites are missing.

Interactive house visualization

The house view is the visual anchor, not a static hero image. Rooms and devices should respond to clicks, focus and state changes. Selecting the hallway light should reveal its current modeled state, controlling routine, relevant saved requirements and recent events. During replay, the selected device should visibly change only as the deterministic simulation applies events.

- Show device state using an icon plus text/status—not color alone.
- Allow a user to select a room or device to filter the timeline and related requirements.
- Highlight the device involved in a failed assertion and draw a restrained connector to the event or requirement details.
- Include a clear legend and avoid implying that the visual model is a live digital twin when it is simulated.

- Do not make 3D polish a substitute for functional correctness; if a real-time 3D house is not feasible, use a crisp interactive SVG/2D floor plan with the same interaction model.

Typography and content rules

- One primary headline per screen. Use concise labels but do not make critical explanation vague.
- Keep status labels explicit: PASS, FAIL, INCONCLUSIVE, SIMULATED, NEEDS REVIEW.
- Use short explanatory sentences near the result; provide deeper event details in an expandable trace.
- Never rely on a tiny tooltip to explain the USP or why a test failed.
- Keep the same type scale and component rhythm across the overview, replay lab and USP/why SWITCH page.

5. Page-by-page product experience

Page 1 — Home overview

Purpose: show the modeled home, current device/routine state, and the most relevant verification status. This is the immersive entry point, not the place to bury the USP in metrics.

- Large title: “Your home, with changes you can verify.”
- House visualization with selectable rooms and devices.
- Right rail with active routines, saved requirements at risk, latest replay status and recent activity.
- Primary CTA: “Open Replay Lab” or “Run regression check.”
- Persistent environment label: “SIMULATED DEMO” until real integration exists.

Page 2 — Why SWITCH / USP

This page must explain the difference with a concrete example, not vague phrases such as “AI-powered safety” or “smarter automation.” It should be a visual walkthrough of one correction becoming a future regression test.

Headline: “A fix today should not become a failure tomorrow.”

35. Household correction — show the exact user statement and context.
36. Confirmed requirement — display the structured rule and its version, with the user-confirmed status.
37. Routine change — show a before/after diff that removes the occupancy condition.
38. Replay — animate a short event trace: occupancy=true → Goodnight fires → hallway light set to off.
39. Failure evidence — identify the failed assertion and the exact event that violated it.
40. Proposed repair — show the guard being restored and explain why it addresses this failure.
41. Regression rerun — show the original requirement and any other saved requirements being tested again.
42. Human decision — the user reviews evidence and approves or rejects the proposed change.

Suggested supporting copy: “SWITCH is designed to remember the corrections your household explicitly confirms. When a related routine changes, it replays those requirements against the new behavior and shows the evidence. That is the product hypothesis—not a claim that conflict detection or regression testing itself is new.”

Page 3 — Replay Lab

Purpose: make the verification engine understandable and inspectable. Use a three-column or split view: routine change, event trace, and assertion results. Provide controls for Run Replay, Apply Suggested Fix, Re-run Suite and Reset Scenario.

- Show a deterministic simulated clock and ordered events.
- Show condition evaluation and state transitions.
- Display each assertion as PASS, FAIL or INCONCLUSIVE, with a reason.
- Show the routine diff separately from the AI’s plain-language explanation.
- Allow the user to inspect which saved requirement triggered the failure.

Page 4 — Saved requirements

Purpose: make the household's durable expectations inspectable and editable. Every requirement should show its source correction, scope, conditions, expected behavior, version, confirmation state and last replay result.

- Users can edit, disable or retire requirements; edits create a new version and preserve history.
- Unconfirmed interpretations remain drafts and do not count as enforced requirements.
- Show stale or untestable requirements explicitly rather than silently skipping them.
- Offer filters by room, device, routine, status and last result.

Page 5 — Reports

Purpose: provide reproducible evidence rather than a vague "safe" score. Include run ID, scenario version, requirement versions, routine version, simulator version, event trace, assertions, proposed diff and reviewer decision.

- Report test counts with PASS / FAIL / INCONCLUSIVE totals.
- Show a timeline with timestamps or simulated ticks and state changes.
- Export a report only if implemented; label demo reports as simulated.
- Never turn a passing result into a blanket home-safety guarantee.

6. End-to-end functional workflow

43. Capture correction: user describes an undesired outcome and relevant context.
44. Interpret: parser or model proposes a structured requirement with entities, conditions, expected state, scope and confidence.
45. Clarify: ask the user about missing or ambiguous conditions.
46. Confirm: user approves the meaning; only then can it become an active requirement.
47. Version: save the confirmed requirement with source, owner, version, timestamps and status.
48. Edit: a user proposes or imports a routine change into a candidate version.

49. Select tests: identify requirements and scenarios affected by the changed routine, plus relevant baseline tests.
50. Replay: execute the candidate against a deterministic simulated state and event sequence.
51. Verify: evaluate explicit assertions and produce PASS / FAIL / INCONCLUSIVE.
52. Explain: show the event trace, violated requirement, relevant condition checks and candidate diff.
53. Suggest: optionally propose a constrained repair. Keep the suggestion separate from verification.
54. Re-run: execute the same tests plus relevant regression requirements against the repaired candidate.
55. Review: user approves, rejects or revises the candidate. No silent deployment.
56. Record: retain the result, versions, trace and human decision for reproducibility.

Result semantics

Status	Meaning	Allowed conclusion
PASS	All required assertions for this run evaluated true in the modeled scenario.	The tested modeled requirements passed for these versions and scenarios.
FAIL	At least one evaluated assertion was violated.	The report must identify the requirement and event evidence.
INCONCLUSIVE	A required condition, device capability, sensor state or model behavior could not be evaluated reliably.	Do not treat as pass; explain the missing evidence.

A missing or unsupported device, unknown state, parser uncertainty, or simulator limitation must not silently turn into PASS. Preserve enough run metadata to reproduce the result.

7. System architecture and trust boundaries

Recommended logical flow: UI → API → requirement service / routine parser → versioned requirement store → deterministic simulator → assertion evaluator →

report service. An optional language model sits beside the deterministic core and proposes structured interpretations or repairs. It does not own the pass/fail decision.

Core components

- Frontend: application shell, house/room visualization, routines editor, requirement library, replay timeline, diff viewer and report view.
- API layer: authentication/household scope, schema validation, version checks, replay job creation and report retrieval.
- Requirement service: draft creation, user confirmation, version history, status transitions and traceability to source correction.
- Routine model: normalized triggers, conditions, actions, targets, priority and version.
- Deterministic simulator: simulated clock, event queue, shared device state, explicit event ordering and recorded transitions.
- Assertion evaluator: executes requirements against traces and returns PASS / FAIL / INCONCLUSIVE.
- Suggestion service: optional LLM or rule-based proposal; schema-validated and sandboxed.
- Report service: stores run configuration, requirement versions, routine versions, assertions, trace and reviewer decision.

Non-negotiable trust boundary

AI can interpret a correction or propose a repair. Only the deterministic verifier can return the test result. Only an authorized human can approve a candidate for deployment.

Replay reproducibility

- Use a fixed scenario seed or explicit event list; do not rely on wall-clock timing for test logic.
- Record routine version, requirement versions, simulator version and scenario input.
- Define event ordering and tie-break behavior in the implementation contract.

- Use identical inputs for before/after comparisons.
- Do not use model-generated text as the assertion engine.

8. Requirement model and API contract

Minimum requirement fields

Field	Purpose
requirement_id	Stable unique identifier.
household_id	Tenant / household boundary.
title	Readable requirement name.
source_correction	User's original correction, retained with appropriate privacy controls.
scope	Relevant room, device, routine or household-wide context.
preconditions	Conditions under which the requirement applies.
expected_behavior	Explicit state/action that must or must not occur.
version	Monotonic version for edits.
status	DRAFT, CONFIRMED, DISABLED or RETIRED.
confirmation	Who confirmed the requirement and when.
last_result	PASS, FAIL or INCONCLUSIVE, plus run reference.

Suggested endpoint shapes

Method / route	Purpose
POST /requirements/drafts	Create a draft from a correction; never auto-confirm.
POST /requirements/{id}/confirm	Confirm the reviewed structured requirement.
GET /requirements	List requirements by household and filter.
PATCH /requirements/{id}	Edit metadata or create a new version; preserve history.
POST /replays	Create a replay for a candidate routine

	version and selected scenarios.
GET /replays/{run_id}	Retrieve run status, assertions and trace.
POST /candidates/{id}/suggest-fix	Request a candidate repair; no deployment.
POST /candidates/{id}/approve	Record authorized human approval; deployment remains a separate, explicit capability.

These are proposed contracts, not a claim that these endpoints are already implemented. Freeze request/response schemas, status enums, error format and version semantics before parallel development.

9. AI/ML strategy

Use the smallest AI capability that improves usability without weakening verification. The project does not need to train a custom deep-learning model to demonstrate its core idea.

Appropriate AI tasks

- Convert a natural-language correction into a draft structured requirement.
- Ask clarifying questions when room, device, condition, scope or expected behavior is ambiguous.
- Summarize why a deterministic assertion failed using the trace as evidence.
- Suggest a constrained candidate edit, with a machine-readable diff and explanation.

Tasks that must remain deterministic

- Event ordering and state transitions in the modeled scenario.
- Evaluation of explicit requirement assertions.
- PASS / FAIL / INCONCLUSIVE assignment.
- Comparison of routine versions and reproducibility metadata.
- Permission checks, confirmation state and approval state.

Validation rules for model output

- Validate structured output against a schema; reject malformed fields and unknown devices/actions.
- Require user confirmation before a draft becomes an active requirement.
- Constrain proposed edits to known routine fields and show the exact diff.
- Re-run the regression suite after a proposed fix; do not infer success from the explanation.
- Provide a deterministic fallback if the model is unavailable.

10. Data and evaluation plan

Start with a small, hand-authored benchmark of representative routines and failure modes. Synthetic scenarios are acceptable for the prototype when clearly labelled; they do not establish real-world effectiveness.

Initial scenario families

- Contradictory writes to the same device.
- Occupancy / presence condition accidentally removed.
- Time-of-day or mode condition changed.
- Event ordering changes the final state.
- One routine fix breaks another previously passing requirement.
- Unknown sensor state or unsupported device capability.
- Ambiguous correction that should remain a draft until clarified.

Evaluation metrics

Metric	How to measure
Regression catch rate	Share of deliberately introduced regressions that cause the correct saved assertion to fail.
False-pass rate	Share of scenarios that violate a requirement but are incorrectly reported as PASS; prioritize minimizing this.
False-fail rate	Share of scenarios that satisfy requirements but are incorrectly marked FAIL.
Replay reproducibility	Whether identical scenario and version inputs yield identical results.

Correction-to-requirement accuracy	Human-reviewed correctness of the structured requirement generated from user language.
User comprehension	Whether participants can identify why a test failed from the trace and report.
Repair verification	Whether a suggested change passes the targeted requirement without breaking the existing regression suite.

Do not invent performance percentages or report accuracy unless they are measured on a documented benchmark. For the demo, use labelled scenario fixtures and report exactly what was tested.

11. Security, privacy and safety boundaries

- Scope every requirement, routine, trace and report to the correct household identity.
- Minimize stored raw household conversation and device data; define retention and deletion behavior before live use.
- Do not send sensitive household details to a third-party model without appropriate disclosure and consent.
- Validate all imported routine definitions and constrain model-generated changes.
- Keep candidate edits separate from live routines; no silent deployment.
- Log approvals and version changes without exposing unnecessary personal information.
- Treat unknown device state as INCONCLUSIVE, not safe.
- State prominently that simulation only validates modeled scenarios and cannot guarantee physical-world safety.

Safety statement for the product

A PASS means the selected modeled requirements passed for the recorded scenario and versions. It does not certify the real home or cover unmodeled sensors, outages, latency, device firmware, or household circumstances.

12. Technology and implementation sequencing

Pragmatic stack

- Frontend: React / Next.js or the existing prototype's current stack; TypeScript where practical; accessible component primitives and a consistent design-token system.
- Backend: Python FastAPI or a Node API—choose one for the MVP and avoid duplicate API layers.
- Persistence: PostgreSQL for households, requirements, versions, routines, replay runs and reports; local demo state is acceptable only when clearly labelled.
- Replay core: a pure deterministic module with unit tests, independent from UI and LLM calls.
- AI integration: provider adapter with structured output validation and a deterministic fallback.
- Testing: unit tests for event ordering and assertions; integration tests for requirement confirmation and replay; end-to-end test for the full demo path.

Build order

57. Freeze the data model, requirement semantics, result statuses and event ordering.
58. Make the browser demo's correction → confirm → fail → repair → rerun story reliable.
59. Implement deterministic simulator and assertion tests before connecting an LLM.
60. Add persistence and API contracts; keep draft and confirmed states separate.
61. Build the replay timeline and trace around real simulator output, not hand-coded result text.
62. Upgrade the house visualization only after state interactions and event evidence are coherent.
63. Add optional AI interpretation and constrained repair suggestions.
64. Run a deliberate regression benchmark, record results and refine the claims.

65. Add a real device/platform integration only after access, permissions and test environment are confirmed.

13. Team responsibilities and integration

Workstream	Ownership	Deliverables
A — UX / Frontend	App shell, house/room view, requirements screens, replay timeline, responsive behavior, accessibility.	Design tokens, component contracts, interactive pages, end-to-end UI flow.
B — Deterministic core / Backend	Routine schema, simulator, event ordering, assertion engine, API and persistence.	Unit-tested replay engine, API schemas, migrations, reproducible run reports.
C — AI / Evaluation / Integration	Correction parser, clarification flow, constrained repair proposals, benchmark and report interpretation.	Validated structured output, prompt/schema tests, evaluation scenarios, integration tests.

The team may reassign these roles, but each deliverable must have one accountable owner. Agree on a single shared contract for routine objects, requirement objects, replay inputs, result statuses and trace events.

Minimum trace event

Each event should include a sequence index or simulated timestamp, routine ID/version, action, target device, prior state, resulting state, condition evaluations and source event. Assertions should reference the relevant trace event IDs.

14. Demo script and judging narrative

A 90–120 second demo

66. Open Home Overview. Explicitly show the SIMULATED DEMO label.
67. Select the hallway light and introduce the saved requirement: “Do not turn off the hallway light while occupancy is true.”
68. Open the routine diff and show that the occupancy guard was removed.

69. Click Run Replay. Animate occupancy=true, Goodnight routine fired, light changed to off.
70. Show FAIL with the precise assertion and event trace—not just a red badge.
71. Click Apply Suggested Fix. Show the proposed diff restoring the occupancy guard.
72. Click Re-run Regression Suite. Show the same scenario now passing in the simulator.
73. Show the saved requirement's version and run report, then explain that the user must approve the candidate.
74. Close with the honest USP: "We are testing the hypothesis that turning confirmed household corrections into reusable regression requirements makes routine changes easier to verify over time."

Judge questions to prepare for

- What is different from Alexa Automated Testing Service? Answer with the consumer-facing correction capture, confirmation, versioned requirement and replay workflow as the hypothesis—not a categorical claim that ATS cannot do any similar thing.
- Is it actually connected to Alexa? State the current prototype status exactly. Do not imply integration that is not implemented.
- How do you know the fix is safe? You do not guarantee real-world safety; you show deterministic evidence for modeled requirements and scenarios.
- Why use AI? To interpret human language and suggest constrained edits, not to decide test outcomes.
- What happens if the correction is ambiguous? Ask a clarifying question and keep it as a draft until confirmed.
- What proves the USP? A comparative prior-art review plus user validation and benchmark evidence; the current audit is preliminary.

15. Risks, open questions and definition of done

Risk	Mitigation
USP overlaps with existing tools	Continue prior-art audit; make claims

	narrower and evidence-based.
Pretty UI hides a weak core	Drive the timeline and report from deterministic replay output.
AI misinterprets a correction	Schema validation, clarification, user confirmation and fallback.
False PASS due to missing data	Use INCONCLUSIVE for unknown or unsupported conditions.
Nondeterministic event order	Define tie-break rules and test them.
Prototype appears live	Persistent SIMULATED DEMO badge and explicit copy.
Fix repairs one test but breaks another	Re-run the relevant saved regression suite.
Scope expands into a full smart-home platform	Keep the MVP centered on correction capture, replay, evidence and approval.

Definition of done for the MVP

- A user can create a correction draft, review its structured interpretation and explicitly confirm it.
- Confirmed requirements have stable IDs and version history.
- A changed routine can be replayed against saved requirements in a deterministic model.
- The replay report includes inputs, versions, ordered trace events and assertion results.
- PASS, FAIL and INCONCLUSIVE have documented semantics.
- A suggested repair is displayed as a diff and is not treated as verified until rerun.
- A passing repair is tested against the relevant regression suite, not just one assertion.
- The UI uses legible typography, accessible contrast, visible focus and meaningful state-driven motion.
- The interface labels simulated data and does not imply physical device control.
- The demo can be reset and replayed consistently.
- The USP language remains qualified until prior-art and user validation are completed.

Final product principle

Build a verifiable workflow first; make it beautiful without letting visual polish imply capabilities the system does not have.

16. Research basis and links

The following sources informed the preliminary competitive positioning. They do not constitute an exhaustive prior-art search or legal novelty opinion.

- Amazon Alexa Automated Testing Service — https://developer.amazon.com/docs/alexaplus/smarthome/automated-testing-service.html
- Amazon ATS: Create test plans — https://developer.amazon.com/docs/alexaplus/smarthome/ats-create-test-plans
- TAPFixer research — https://arxiv.org/abs/2407.09095
- SHACR research — https://arxiv.org/abs/2606.22312
- Hirz repository — https://github.com/BashaarJavaid/Hirz
- User-centric evaluation of smart-home conflict resolution — https://experts.illinois.edu/en/publications/a-user-centric-evaluation-of-smart-home-resolution-approaches-for/
- Alexa+ add-ons — https://developer.amazon.com/docs/alexaplus/add-ons/home
- Alexa+ MCP toolkit quickstart — https://developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-quickstart

Research conclusion

The preliminary review indicates that generic conflict detection, repair, natural-language test plans and regression testing are not defensible as unique by themselves. The proposed angle to validate is the consumer-side process of turning explicitly confirmed household corrections into contextual, versioned, reusable regression requirements across future edits. More research and user testing are required before claiming that this workflow is unique.

SWITCH

ALEXA+ INTEGRATED PRODUCT, UX & TECHNICAL MASTER SPECIFICATION

Smart-home routine verification, replay and regression requirements — extended for an Alexa+ experience

SINGLE SOURCE OF TRUTH FOR PERSON 1 • PERSON 2 • PERSON 3

Preserve the existing SWITCH product and core USP. Add a coherent Alexa+ interaction/integration layer without claiming capabilities that have not been implemented and tested.

Version: 2.0 — Alexa+ adaptation | **Date:** 22 October 2026

This is an implementation blueprint, not a claim that every described capability is already implemented. Teams must mark features IMPLEMENTED, SIMULATED, BLOCKED or NOT IMPLEMENTED and attach evidence.

Document control and how to use this specification

This document preserves the existing SWITCH Updated Product, UX & USP Master Document and extends it for the Amazon Alexa+ hackathon. It is the shared contract for three parallel contributors. Alexa+ integration details are proposed until the selected route is implemented and tested; the original prototype must not be described as already connected.

Field	Decision
Product	SWITCH — a correction-to-regression workflow for smart-home routines.
Primary orientation	Alexa+ experience; select and implement one supported integration path.
Core USP to preserve	Confirmed household correction → versioned requirement → deterministic replay → evidence → human approval.
UI baseline	Existing seven-page shell, premium interactive house, device panel, replay tools, saved requirements, reports and settings.
Trust boundary	AI may interpret or suggest; deterministic evaluator owns PASS / FAIL / INCONCLUSIVE.
Original baseline from source PDF	Prototype simulates devices/routines/results and is not connected to a physical device, Amazon account, live Alexa services, configured backend, database or live LLM.
Audience	Person 1: UX/frontend/house viewer; Person 2: deterministic core/backend; Person 3: AI/evaluation/Alexa+ integration.

NON-NEGOTIABLE PRESERVATION RULE

Do not rebuild SWITCH from scratch, remove working features, replace the house with a placeholder, or discard tests/docs. Inspect the current ZIP/repository first, make a backup/branch, run baseline tests, implement additive changes, and rerun tests.

Status label	Meaning
IMPLEMENTED	Code exists and a reproducible test/demo confirms it.
SIMULATED	Local/demo data; no real Amazon/Alexa call is made.
BLOCKED	Depends on access, credentials, preview eligibility or capability not yet available.
NOT IMPLEMENTED	Planned only; do not show as connected or working.

Contents

- 1 Executive summary and pitch
- 2 Preserve vs Alexa+ additions
- 3 User problem and canonical scenario
- 4 USP and positioning guardrails
- 5 Architecture and Alexa+ route selection
- 6 End-to-end flowcharts
- 7 Page-by-page UX specification
- 8 Wireframes and interactions
- 9 3D house, state and day/night
- 10 Shared data models
- 11 Deterministic replay and result semantics
- 12 Alexa+ adapter / MCP / Agent Skill contract
- 13 API contracts and trace schema
- 14 AI boundaries
- 15 Security, privacy and trust
- 16 Visual design, motion and accessibility
- 17 Person 1/2/3 ownership
- 18 Build sequence and integration gates
- 19 Test plan and definition of done
- 20 Demo script
- 21 Risks and judge Q&A;
- 22 References and handoff checklist

Implementation rule

If the PDF differs from existing code, inspect the code and preserve working behavior. Record the discrepancy and update the contract rather than silently deleting functionality.

1. Executive summary and product pitch

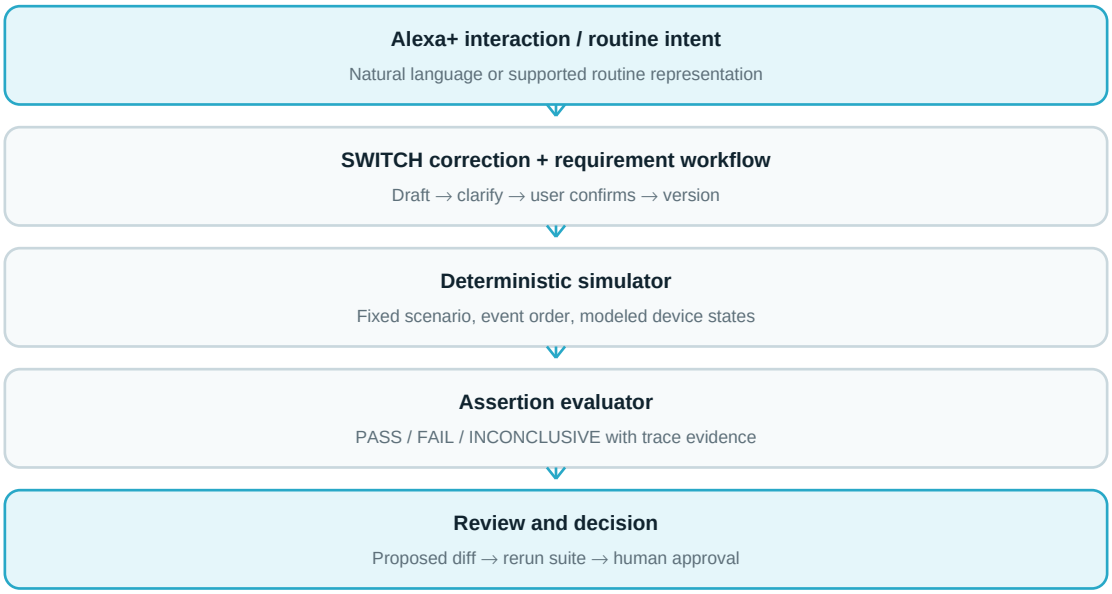
SWITCH is a smart-home routine verification layer. It preserves explicitly confirmed household corrections as contextual, versioned regression requirements and replays them when related routines change. The Alexa+ adaptation makes Alexa+ the primary assistant context for capturing routine intent and representing routines being verified; it does not replace the deterministic SWITCH core.

One-sentence pitch

A fix today should not become a failure tomorrow.
SWITCH helps an Alexa+ household turn a confirmed correction into a reusable test, replay it after a routine edit, and inspect evidence before approving the change.

Example

A user says: “Alexa, don't turn off the hallway light while someone is there.” SWITCH asks for confirmation and stores the rule. Later, an edit removes the occupancy condition from Goodnight. SWITCH replays a scenario with occupancy=true. If the routine turns the light off, the requirement fails. The app shows the action and event trace, proposes restoring the guard, reruns the suite and leaves final approval to the user.



Truthful scope
A passing result means only that selected modeled requirements passed for recorded scenarios and versions. It does not certify the physical home or imply a live Alexa+ connection.

2. What stays unchanged vs what becomes Alexa+-oriented

Preserve from original SWITCH	Add / adapt for Alexa+
Home Overview, interactive house, room/device selection, state overlays.	Alexa+ assistant entry point, integration status and Alexa-oriented routine context.
Why SWITCH page explaining correction → confirmed requirement → future replay.	Show how natural-language Alexa+ interaction becomes a confirmed requirement.
Replay Lab: clock, routine diff, trace, assertions, repair and rerun.	Routine source/adapter provenance; imported/live/simulated labels.
Saved Requirements: source correction, scope, conditions, expected behavior, version, confirmation and result.	Link requirement to Alexa-oriented routine IDs/versions where supported.
Reports with run ID, scenario, versions, trace, assertions and reviewer decision.	Integration mode, adapter version, safe correlation ID and provenance.
Devices / Integrations and Settings.	Alexa+ setup/status, connection test, permission disclosure, supported capability list, disconnect/reset.
Existing tests, documentation, assets and design tokens.	Adapter contract tests, integration smoke tests and fallback path.

Do not turn SWITCH into a generic Alexa device-control dashboard. Its purpose is preserving corrections and verifying future routine changes.

Environment badge behavior

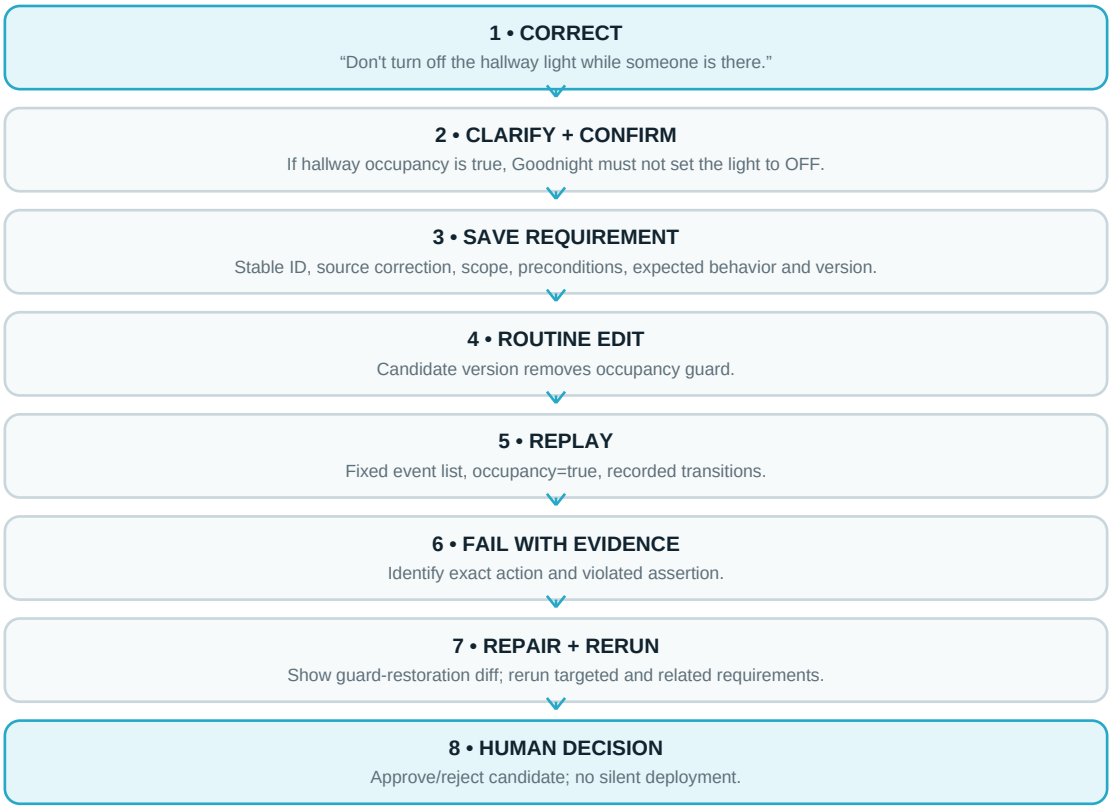
Mode	Visible label	Allowed behavior
Local fixtures	SIMULATED DEMO	Local routines/devices and deterministic scenarios; no live Alexa claims.
Configured but not authenticated	ALEXA+ SETUP / NOT CONNECTED	Show configuration status only.
Verified test connection	ALEXA+ CONNECTED (only after verified success)	Describe the exact scope and supported operations.
Disconnected / unsupported	DISCONNECTED / UNSUPPORTED	Preserve local work; show retry/fallback; never fabricate live state.

3. User problem and canonical scenario

Routines can interact through shared devices, occupancy, time, modes and event order. A routine can work alone but violate a household expectation after a later edit. A manual override does not automatically express a permanent preference; the user must confirm whether the correction should become a durable requirement.

Persona	Need	SWITCH response
Household member	Express a correction without writing test scripts.	Capture statement/context; propose structured draft; ask confirmation.
Routine editor	Know whether an edit breaks earlier expectations.	Compare versions; run relevant suite; inspect evidence.
Reviewer / owner	Retain control over durable rules and changes.	Review proposed diff, replay results/history; approve or reject explicitly.

Correction-to-regression workflow



4. USP and positioning guardrails

The differentiating hypothesis is a consumer-facing correction-to-regression workflow: an explicitly confirmed household correction becomes a contextual, reviewable, versioned requirement that can be replayed against later routine edits.

Generic capability	SWITCH proposed focus
Detect a conflict once	Preserve a confirmed correction as a reusable requirement.
Show a suggested fix	Show failed expectation, trace, exact diff and rerun result.
Use AI to explain/generate a change	AI interprets/suggests; deterministic checks own pass/fail.
Test a current scenario	Replay saved requirements after related routine edits.
Promise home safety	Report exactly which modeled requirements passed, failed or were inconclusive.

Claims we must not make

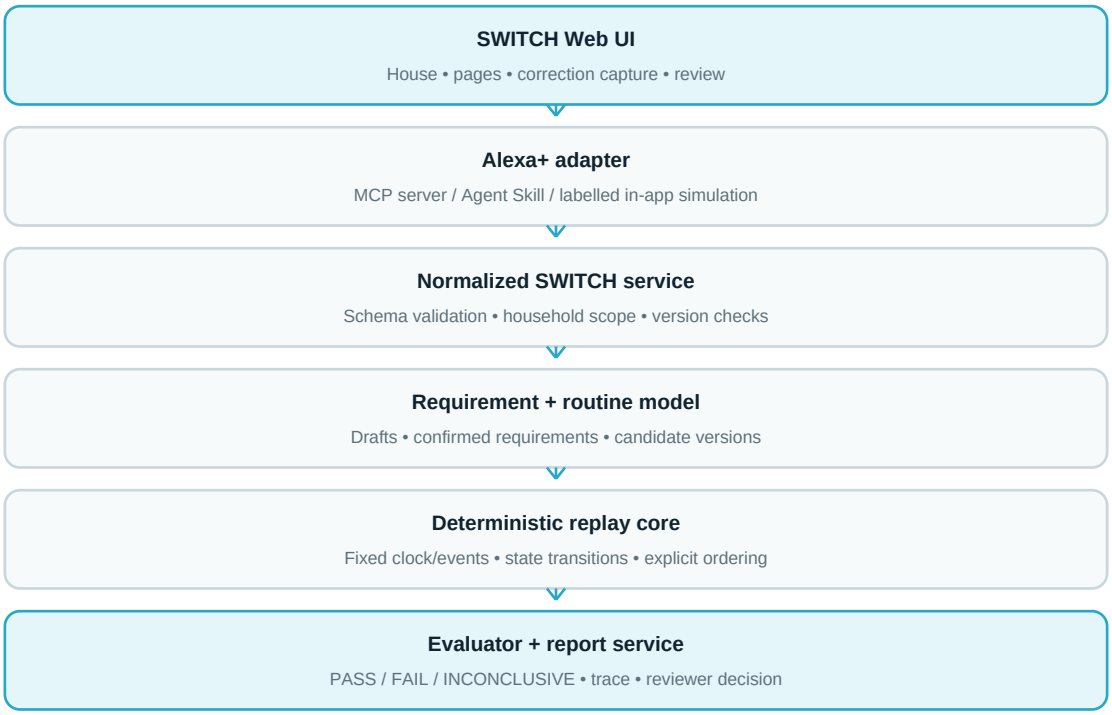
- Do not claim “world's first,” “guaranteed safe,” or “prevents all conflicts.”
- Do not claim existing Amazon testing tools cannot do similar work; competitive review is preliminary.
- Do not claim live Alexa+ integration until implemented and tested with supported access.
- An AI-generated fix is not verified until deterministic tests rerun.
- Do not equate a conversational response with a device command being executed or a physical state being measured.

Validation plan

- Compare correction capture, confirmation, persistence and cross-edit replay with relevant Alexa testing and smart-home automation tools.
- Interview 5–10 smart-home users about repeated routine failures and value of preserving a correction as a test.
- Measure regression catch rate, false-pass/false-fail rates, reproducibility, interpretation accuracy and user comprehension.
- Record fixtures and measured results; never invent performance percentages.

5. Architecture and Alexa+ route selection

The UI calls a SWITCH service. The requirement service stores confirmed rules and versions. The deterministic simulator and assertion evaluator execute repeatable scenarios. Reports retain evidence. An Alexa+ adapter translates supported input/routine representations into the internal contract; it never decides PASS/FAIL.



Select one primary Alexa+ route

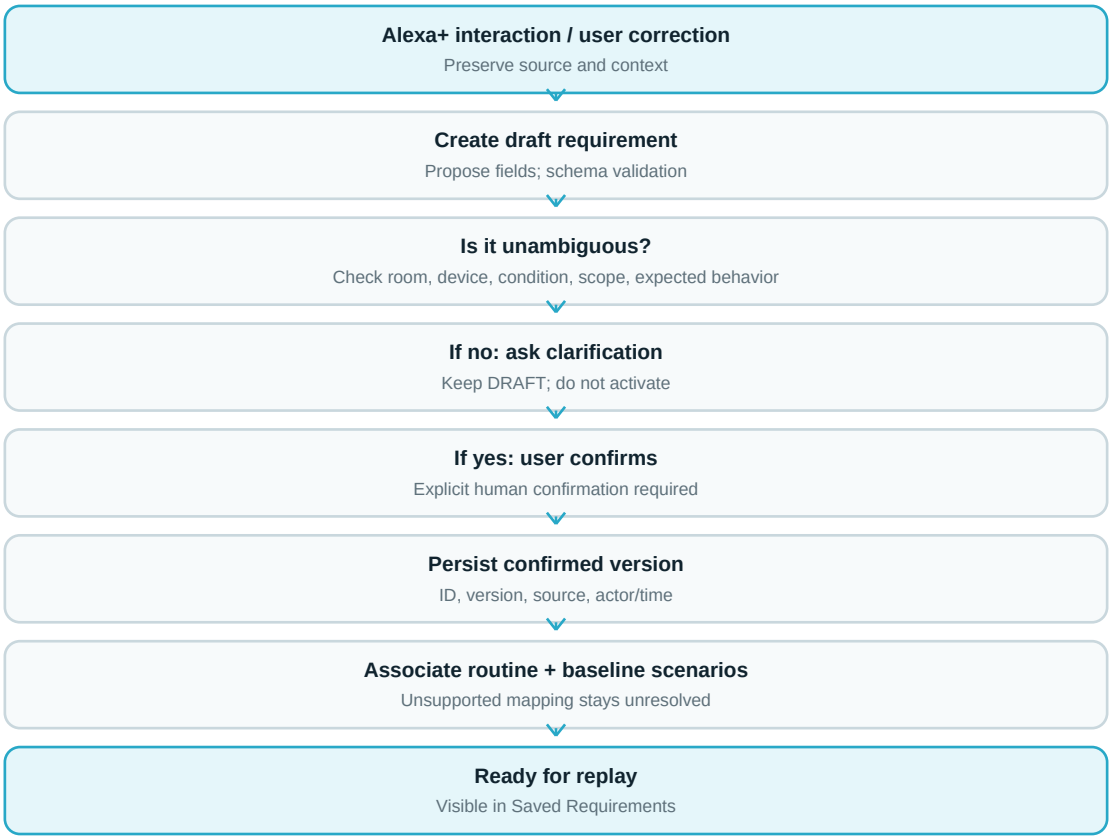
Route	Build	When / proof
A • MCP server	Self-hosted MCP server with Streamable HTTP and MCP spec 2025-11-25 or later, per hackathon requirement.	Choose when eligible access is available. Demo a supported Alexa+ interaction invoking a real tool; include source, configuration and test evidence.
B • Agent Skill	A supported Alexa+ Agent Skill exposing the SWITCH workflow.	Choose if the documented skill route and access are available. Demo it invoked and show a meaningful workflow.
C • Simulated Alexa+ web experience	In-app Alexa+ interaction connected to real SWITCH code and deterministic simulator.	Allowed fallback if live preview/access is unavailable. Include simulation source code, demo it working, and label SIMULATED DEMO.

Decision gate

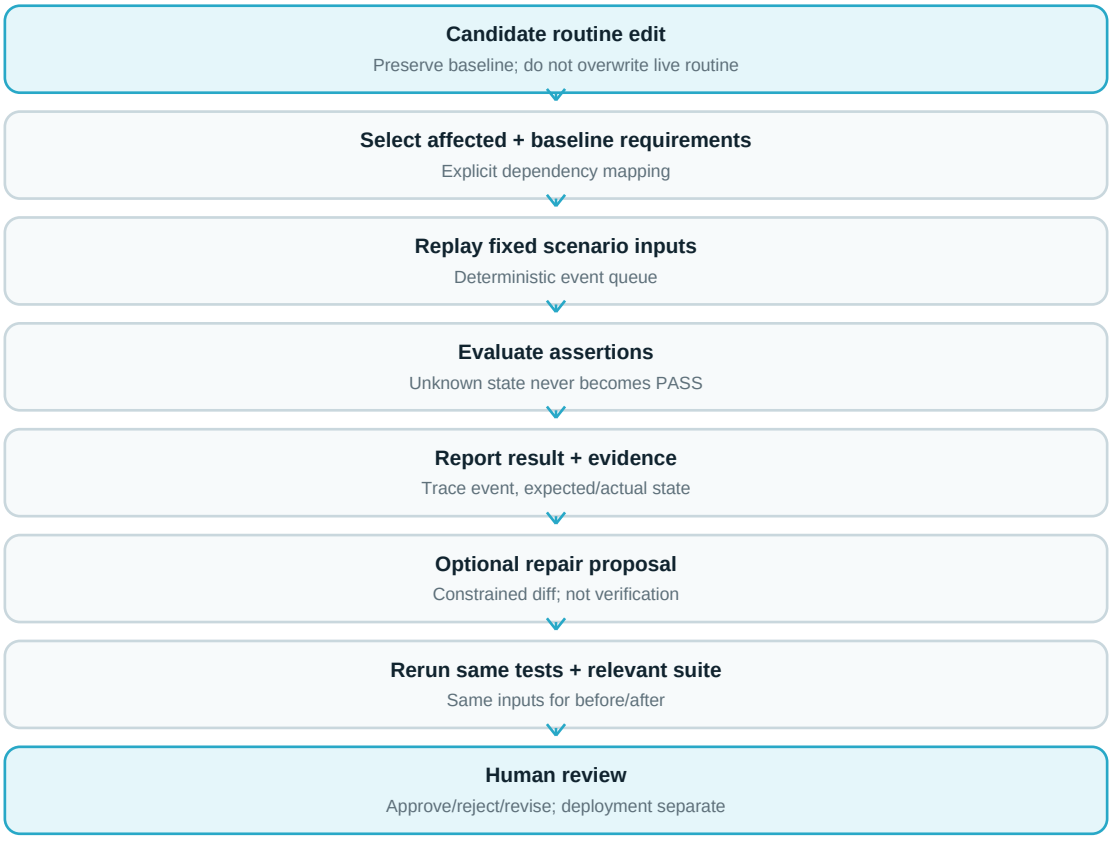
Person 3 verifies current access, official requirements, auth/transport expectations and testing environment before choosing. Do not implement multiple routes in parallel unless the primary route already works.

6. End-to-end flowcharts

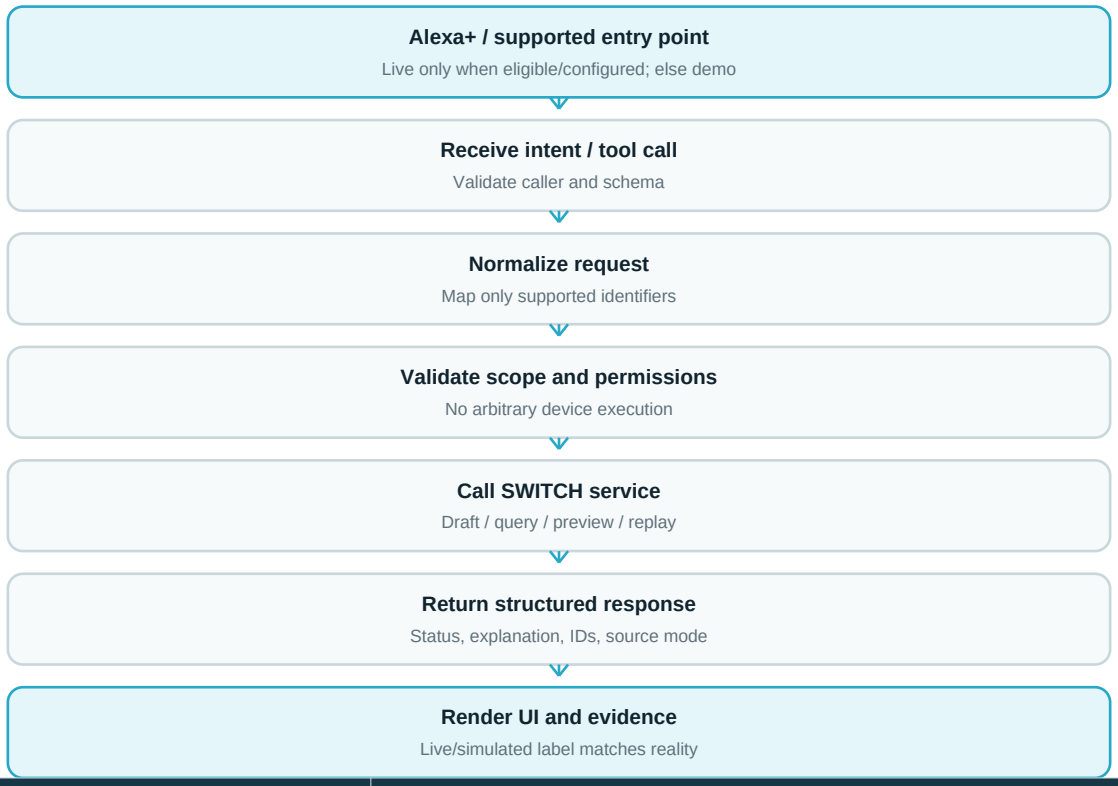
6.1 Correction → confirmed requirement



6.2 Routine change → verified report



6.3 Alexa+ adapter sequence and failure branches



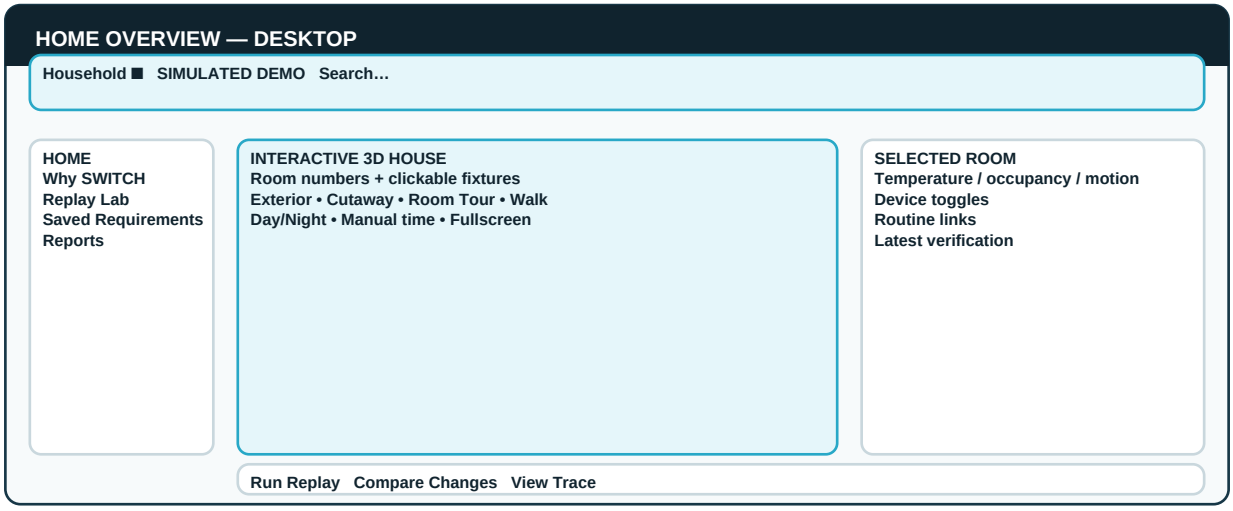
Condition	Required behavior
No live access	Use source-included simulation; show SIMULATED DEMO.
Unknown room/device/routine	Reject or clarify; never silently remap.
Unsupported capability / unknown sensor	Return UNSUPPORTED or INCONCLUSIVE with reason.
Timeout / network failure	Preserve local work; show disconnected state; allow retry.
Auth failure	Actionable error; never expose secrets.
AI unavailable	Use deterministic templates if possible; otherwise leave draft or return inconclusive.

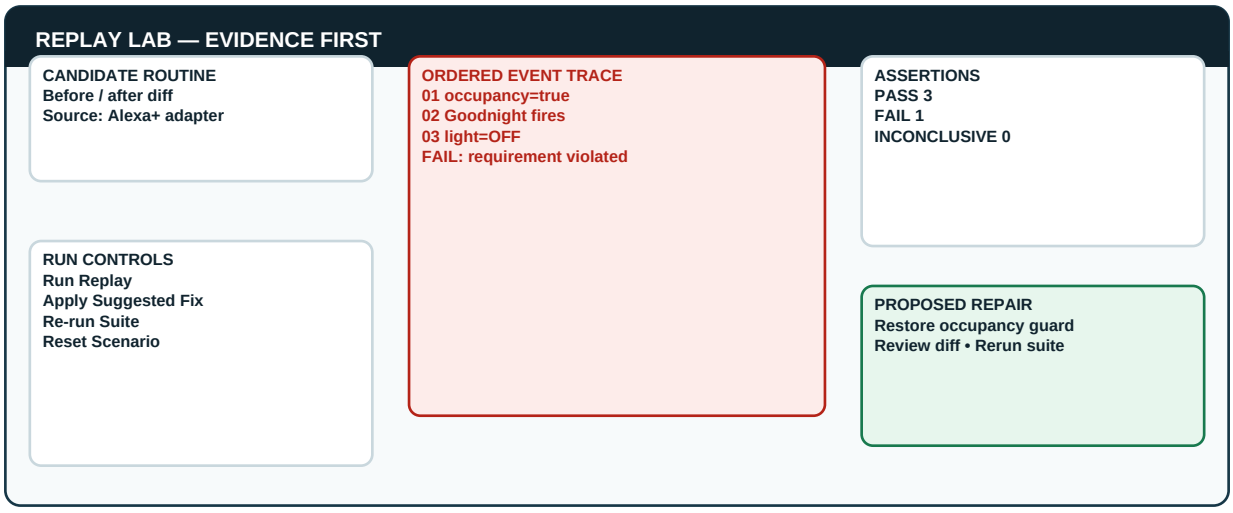
7. Page-by-page product experience

Page	Required content	Alexa+ adaptation
1 Home Overview	Interactive house, room/device state, active routines, requirements at risk, latest replay, recent activity, Run Regression CTA.	Household selector, integration badge, assistant entry point; same state drives house and panels.
2 Why SWITCH / USP	Correction → confirmation → diff → replay → failure evidence → repair → rerun → human decision.	Use Alexa+ conversational context; label simulated vs live.
3 Replay Lab	Routine change, event trace, assertions, deterministic clock, Run Replay, Apply Fix, Rerun Suite, Reset.	Show routine source/provenance and normalized diff; transcript is not test evidence.
4 Saved Requirements	Source correction, scope, conditions, expected behavior, version, confirmation, status, last result; edit/disable/retire with history.	Store original utterance only with suitable privacy controls; show mapping status.
5 Reports	Run ID, scenario, versions, trace, assertions, proposed diff and reviewer decision.	Include integration mode, adapter version, safe correlation ID and provenance.
6 Devices / Integrations	Devices, capabilities, room, modeled state, health and supported operations.	Alexa+ route, connection state, last health check, permissions, supported tools, test/disconnect.
7 Settings	Household/profile, accessibility, retention, reset demo and environment.	Integration setup, secret handling, consent/privacy, simulated-data controls.

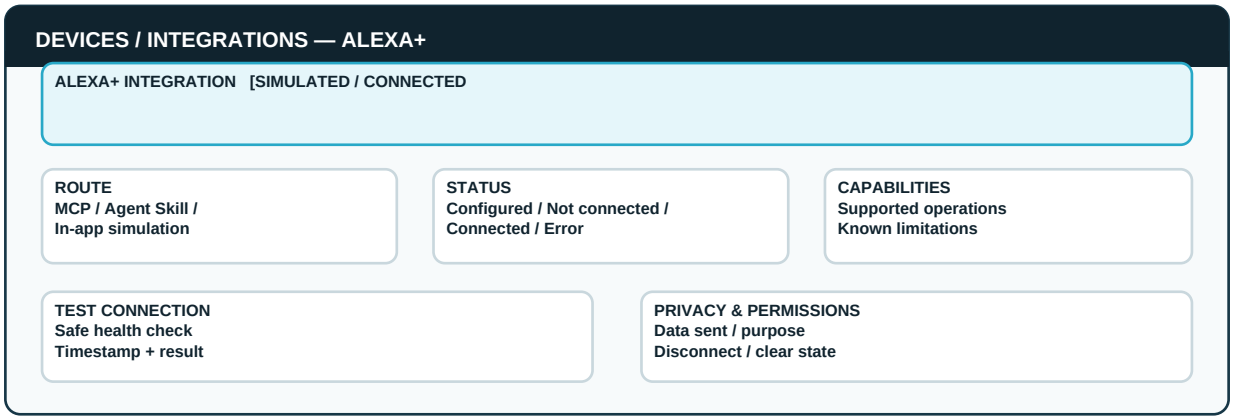
8. Wireframes and interaction details

Wireframes are layout contracts, not final artwork. Preserve the existing dark slate/cyan palette, readable type, and working navigation. Extend existing components instead of building a disconnected mini-app.





8.1 Alexa+ integration page wireframe



8.2 House viewer interaction requirements

- The 3D house is the visual anchor, not a static image. Rooms and devices are selectable and map to the same state used by side panels, timeline and assertions.
- Provide Exterior, Cutaway, Room Tour, Walk, previous/next room, numbered markers, fullscreen enter/exit, reset camera and exit-to-exterior.
- Room tour must put the camera inside the selected room at plausible eye height and avoid walls/floors/ceilings. Use asset-derived bounds and collision checks, not guessed coordinates alone.
- If the supplied GLB lacks usable interior geometry, report the limitation; do not pretend the room is accessible.
- Room numbering must match the shipped asset. Candidate list: 1 Living room, 2 Kitchen, 3 Bedroom, 4 Bathroom, 5 Hallway, 6 Bedroom 2, 7 Bedroom 3, 8 Patio—verify against the model.
- Day/night defaults to local browser time. Manual clock/slider overrides until Reset to local time. This is scene lighting, not a claim about physical lights.
- Use purposeful camera transitions, device feedback, replay stepping and failure highlights. Respect reduced-motion preferences.

9. 3D house, canonical state and day/night

The viewer and simulator must share one canonical application state. A toggle must update modeled state and visual representation; the UI must never imply a physical device changed unless that capability was actually verified.

Entity	Minimum state	UI behavior
Room	room_id, name, floor, bounds, camera preset, device IDs	Numbered clickable markers; room tour; selected room panel; preset verified against mesh bounds.
Device	device_id, room_id, type, capability, modeled state, provenance	Selectable fixture/hotspot; icon plus text; toggles only for supported modeled capabilities.
Sensor	sensor_id, room_id, type, value, tick/time, source	Label virtual/simulated sensor values; unknown stays unknown.
Routine	routine_id, name, version, trigger, conditions, ordered actions, source	Show baseline/candidate versions and human-readable diff.
Scene clock	local timezone, auto/manual mode, time, day/night state	Auto follows local time; manual mode stays fixed until reset.
Replay run	run_id, scenario seed/input, versions, trace, assertions, status	Same data drives animation, trace and report.

Camera and navigation acceptance

- Room click validates room ID, animates to an authored preset, checks camera target against bounds and prevents camera inside geometry.
- Room arrows wrap or disable consistently and show index such as 03 / 08.
- Fallback to safe room/exterior view if preset/collision data is unavailable.
- Fullscreen must synchronize with browser fullscreenchange; test enter, exit and resize.
- Model load errors show expected path and recovery action. Confirm the GLB exists at the exact path before build.

Day/night behavior

Control	Expected behavior
AUTO (default)	Use browser local time; derive scene daylight/night lighting; show local time and AUTO.
Manual clock	User-selected time updates lighting immediately; show MANUAL and selected time.
Reset to local time	Return to auto mode and current local time.
Timezone / midnight	Handle local timezone and midnight; don't claim precise solar position unless implemented.

10. Shared data models and lifecycle

Use the current repository's equivalent schema where it exists. Add fields through backward-compatible changes; do not create duplicate competing models.

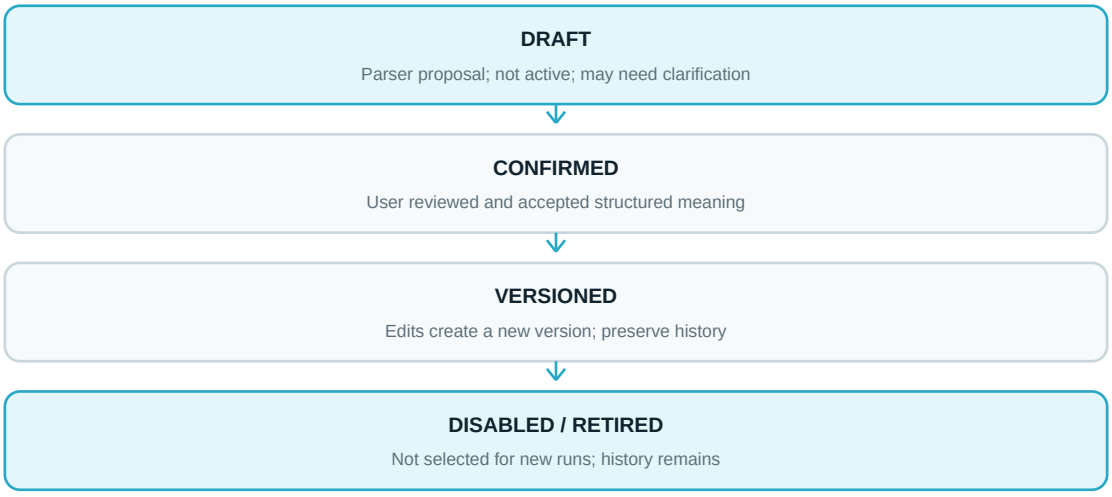
Confirmed requirement — illustrative JSON

{
"requirement_id": "req_hallway_occupancy_001",
"household_id": "demo_household",
"title": "Keep hallway light on while occupied",
"source_correction": {
"text": "Don't turn off the hallway light while someone is there",
"source": "alexa_plus_interaction",
"mode": "simulated_or_live_as_verified"
},
"scope": {"room_id": "hallway", "device_id": "hallway_light", "routine_id": "goodnight"},
"preconditions": [{"entity": "hallway_occupancy", "op": "==", "value": true}],
"expected_behavior": {"device": "hallway_light", "state_not": "OFF"},
"version": 1, "status": "CONFIRMED",
"confirmation": {"actor": "household_user", "timestamp": "ISO-8601"},
"last_result": "INCONCLUSIVE"
}

Routine candidate — illustrative JSON

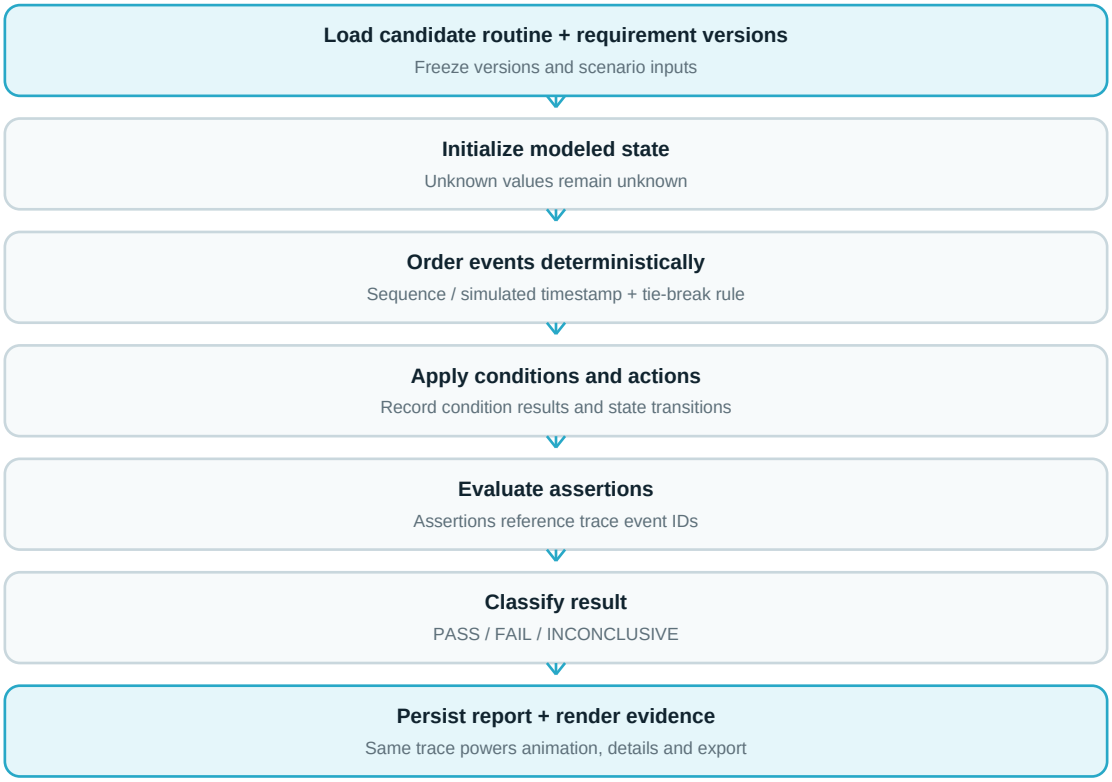
{
"routine_id": "goodnight", "version": 4,
"source": "alexa_adapter_or_demo_fixture",
"trigger": {"type": "user_intent", "name": "Goodnight"},
"conditions": [{"entity": "hallway_occupancy", "op": "==", "value": false}],
"actions": [{"sequence": 1, "target": "hallway_light",
"capability": "power", "operation": "set", "value": "OFF"}]
}

Requirement lifecycle



11. Deterministic replay and result semantics

Replay must be reproducible. Use explicit scenario inputs or fixed seed, simulated clock, deterministic event queue and documented tie-break rules. Keep the replay module independent of UI rendering and AI calls.



Result semantics

Status	Meaning	Allowed conclusion
PASS	All required assertions evaluated true in this modeled run.	Only these requirements passed for these versions and scenarios.
FAIL	At least one evaluated assertion was violated.	Report requirement ID, event evidence and expected/actual state.
INCONCLUSIVE	Required condition, capability, sensor state or model behavior could not be evaluated reliably.	Not a pass; explain missing evidence.

Non-negotiable rules

- Unknown sensor state, unsupported capability, parser ambiguity or missing mapping must never silently become PASS.
- An AI explanation is not evidence; the trace and assertion evaluator are evidence.
- An AI repair is a candidate diff, not a verified fix until rerun.
- After repair, run targeted tests and relevant regression requirements.
- Candidate changes are not deployed automatically; explicit approval is required, and deployment is a separate capability.

12. Alexa+ adapter / MCP server / Agent Skill contract

Implement the official supported contract confirmed by Person 3. The hackathon instructions describe an Alexa+ MCP route using a self-hosted server with Streamable HTTP and MCP specification 2025-11-25 or later, or an Agent Skill. They also allow an in-app simulated Alexa+ experience if its source is included and the demo clearly shows it working.

Proposed tool	Purpose	Safety / response requirement
switch_create_correction_draft	Create draft from natural language and context.	Returns draft ID, proposed fields and ambiguities; never auto-confirms.
switch_get_requirement	Read a requirement or history.	Household-scoped; return only needed data.
switch_confirm_requirement	Confirm reviewed draft.	Requires explicit confirmation; records actor/time/version.
switch_list_routines	List supported normalized routines/versions.	Only data actually available; show simulated/imported/live provenance.
switch_preview_routine_change	Return candidate diff and affected requirements.	No live change; validate fields and scope.
switch_run_replay	Run deterministic tests.	Return run ID, versions, counts and trace reference.
switch_get_replay_report	Retrieve assertions and trace.	Trace-backed; never infer PASS from generated text.
switch_propose_repair	Create constrained candidate diff.	No automatic application to a live routine.
switch_record_review_decision	Record approve/reject/revise.	Human decision separate from deployment.

Implementation notes

- Use narrow, typed tools with input/output schemas; reject unknown fields, IDs and unsupported actions.
- Verify official transport, protocol, auth and preview requirements before coding.
- Keep adapter separate from deterministic core so the same tests run on simulated or supported live input.
- Store credentials outside source control; never commit keys, bearer tokens or preview credentials.
- Add a harmless health/status operation and a connection-test UI.
- Log safe operational metadata only; avoid full household conversation logging.
- Document local setup, endpoint, authentication, supported operations and known limitations.

13. API contracts and event trace

These are proposed contracts, not claims that routes already exist. Reuse the current API framework and avoid duplicate API layers.

Method / route	Purpose
POST /requirements/drafts	Create draft; never auto-confirm.
POST /requirements/{id}/confirm	Confirm reviewed structured requirement.
GET /requirements	List/filter by household, room, device, routine, status, result.
PATCH /requirements/{id}	Edit metadata or create a version; preserve history.
POST /routines/candidates	Create candidate routine version; keep baseline.
GET /routines/{id}/diff?from=&to;=	Return validated before/after diff.
POST /replays	Run candidate against selected scenarios.
GET /replays/{run_id}	Retrieve status, assertions, trace and versions.
POST /candidates/{id}/suggest-fix	Request constrained repair; no deployment.
POST /candidates/{id}/review	Record approve/reject/revise.
GET /integrations/alexa/status	Route, connection state, capabilities and last health check.
POST /integrations/alexa/test	Harmless integration health check; no device control.

Minimum trace event — illustrative JSON


```
{
"event_id":"evt_0003", "sequence":3, "simulated_time":"00:00:03",
"routine_id":"goodnight", "routine_version":4, "action":"set",
"target_device_id":"hallway_light", "prior_state":"ON", "resulting_state":"OFF",
"condition_evaluations":[{"condition":"hallway_occupancy == false",
"actual":true,"result":"PASS"}],
"source_event_id":"evt_0002"
}
```


Every assertion references trace event IDs. Each report includes run ID, scenario input/seed, requirement versions, routine version, simulator version, integration mode/provenance, assertions, trace, proposed diff and reviewer decision.

14. AI interpretation and repair boundaries

AI may do	Must remain deterministic / human-controlled
Convert correction into draft structured requirement.	Event ordering, state transitions and replay.
Ask clarifying questions about room/device/condition/scope.	Assertion evaluation and PASS / FAIL / INCONCLUSIVE.
Summarize failure using trace evidence.	Version comparison and reproducibility.
Suggest constrained candidate edit with machine-readable diff.	Permissions, confirmation, approval and deployment.
Explain supported integration limitations.	Any physical device execution; never silent.

Validation rules

- Validate model output against schema; reject malformed fields and unknown devices/actions.
- Require user confirmation before a draft becomes active.
- Constrain proposals to known routine fields and show exact before/after values.
- Rerun regression suite after proposed fix; do not infer success from explanation.
- Use deterministic fallback if model unavailable.
- If intent is ambiguous, ask a question and keep DRAFT status.

15. Security, privacy, consent and trust

- Scope requirements, routines, traces and reports to the correct household identity.
- Minimize raw voice/text and device-history storage; define retention/deletion before live use.
- Disclose what is sent to Alexa+ or third-party models; obtain appropriate consent.
- Validate imported routines and constrain generated changes; treat tool input as untrusted.
- Keep candidate edits separate from live routines; no silent deployment.
- Keep secrets out of source control and redact them from logs.
- Unknown sensor/device state is INCONCLUSIVE, not safe.
- Persistent SIMULATED DEMO badge until real integration is verified.
- A modeled PASS does not cover unmodeled sensors, outages, latency, firmware or household circumstances.

Safety statement

A PASS means the selected modeled requirements passed for recorded scenarios and versions. It does not certify the real home or guarantee physical-world safety.

Integration status fields

Field	Required values / meaning
integration_mode	simulated_demo | mcp | agent_skill
connection_status	not_configured | disconnected | connected | error
source_provenance	local_fixture | imported_routine | verified_live_source
capabilities	Explicit list of operations confirmed to work.
last_health_check	Timestamp + success/failure + safe diagnostic.
user_consent	Record consent for turning correction into durable requirement.

16. Visual design, motion and accessibility

Keep the premium interactive smart-home feel: graphite/deep slate surfaces, restrained blue/cyan highlights, warm neutral text, semantic status colors, generous spacing and legible typography. Avoid pure-black walls of UI and same-weight card grids.

Area	Specification
Typography	Desktop body 16–18 px where layout permits; labels 14–16 px; section headings 22–28 px; main title 34–44 px. Avoid tiny essential labels.
Layout	Clear hierarchy, generous spacing, consistent radii, subtle borders and aligned controls.
Motion	Short transitions for camera travel, replay steps, device changes, timeline events and results. Respect reduced-motion preferences.
Accessibility	Keyboard navigation, visible focus, contrast, semantic controls, accessible icon labels and non-color status cues.
Responsive	House view collapses to overview + stacked panels on narrow screens; no horizontal overflow.
Errors	Readable model/integration errors, retry/back-to-safe-view; no endless spinner without explanation.
Evidence	Status text, requirement ID and event details; never color-only or tooltip-only explanation.

State-driven motion checklist

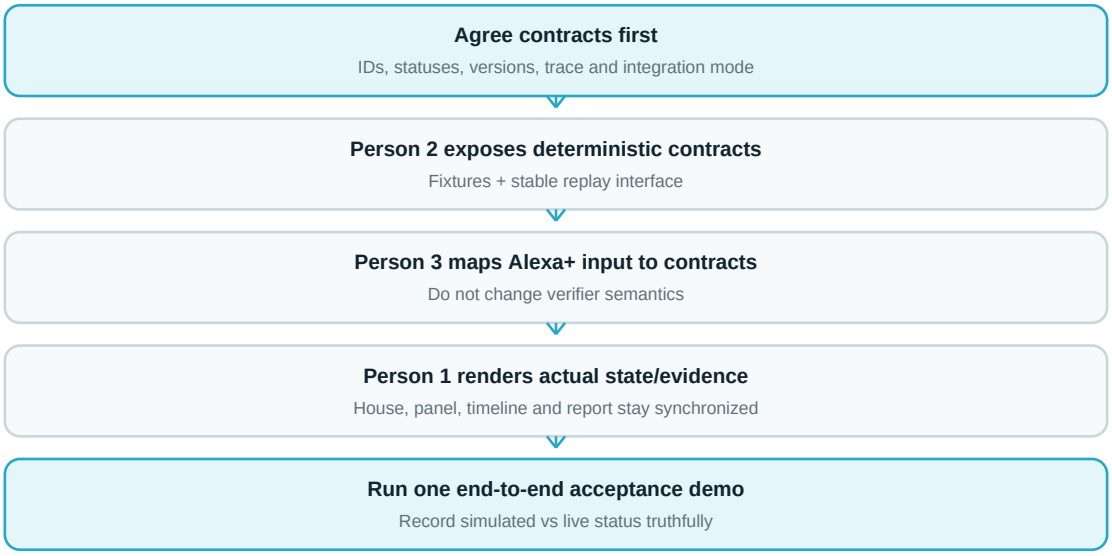
- Selecting a room moves to a verified interior preset and updates the room panel.
- Selecting a device highlights it and synchronizes side panel/timeline.
- Replay animates only when deterministic simulator applies events.
- FAIL highlights the target device and links to assertion/trace.
- Applying a fix changes the candidate diff; results update only after verifier reruns.
- Day/night follows auto local time or manual time; mode remains visible.
- Test fullscreen enter/exit, resize, keyboard navigation and room switching.
- Reduced-motion mode disables nonessential transitions.

17. Person 1 / Person 2 / Person 3 ownership

Each workstream has one accountable owner. Share one contract for room/device IDs, routine and requirement objects, replay inputs, result statuses, trace events and integration provenance.

Owner	Owns	Deliverables	Must not
PERSON 1 — UX / Frontend	App shell, seven pages, house/room viewer, state bindings, motion, time controls, responsive UI.	Interactive room/device view; safe camera presets; accessible UI; explicit integration badge; screenshots and end-to-end UI flow.	Fake Alexa connection; hardcode results disconnected from simulator; replace house with placeholder.
PERSON 2 — Deterministic core / Backend	Routine schema, lifecycle/versioning, simulator, event order, assertions, API and persistence if feasible.	Unit-tested replay; schemas; reproducible reports; status semantics; contract tests.	Use AI as verifier; mark unknown PASS; silently deploy candidate changes.
PERSON 3 — AI / Evaluation / Alexa+	Correction parser, clarification, schema validation, repair proposals, route selection and adapter.	MCP/Agent Skill or source-included labelled simulation; tool schemas; setup docs; health checks; evaluation tests.	Claim live integration without proof; auto-confirm ambiguity; invent unsupported API methods.

Shared integration handshake



18. Build sequence and integration gates

Gate	Work	Exit criteria
G0 Preserve baseline	Inspect ZIP/repo; note stack; backup/branch; run tests/build; record screenshots.	Recoverable baseline and test results.
G1 Freeze contracts	Agree IDs, schemas, statuses, trace and version rules.	One shared contract; no duplicate models.
G2 Stabilize core story	Correction → confirm → fail → repair → rerun using fixtures.	Deterministic repeatable result and trace.
G3 Bind house UI	Canonical room/device state; camera, collision, fullscreen, day/night.	House/panels synchronized; camera acceptance passes.
G4 Select Alexa+ route	Verify official requirements and access; choose one route.	Decision recorded with setup steps and fallback.
G5 Implement adapter	Typed tool/skill schemas, health/status and validation.	Contract tests pass; unsupported requests rejected.
G6 End-to-end integration	Alexa-oriented correction → confirm → replay → fail → repair → rerun.	One reproducible demo; integration mode visible.
G7 Package	Unit/integration/browser/accessibility/responsive/export checks; docs and ZIP.	Clean extract/install/test/build succeeds; known gaps documented.

Do not block the entire team on live Alexa access. Person 1 and Person 2 can work against stable simulated contracts while Person 3 verifies the real route. The fallback must be a real, source-included simulated Alexa+ experience, not an unlabeled mock.

19. Test plan and definition of done

Test group	Minimum acceptance checks
Existing regression	All baseline tests pass; no removed features; existing pages and exports still work.
Requirement lifecycle	Draft not active; confirmation required; edits preserve history; ambiguous input remains draft.
Deterministic replay	Same input gives same trace/result; event ordering tested; unknown sensor gives INCONCLUSIVE.
Canonical failure	occupancy=true + Goodnight guard removed → FAIL with exact light-off event and requirement.
Repair/rerun	Restored guard reruns target and relevant suite; UI derives result from evaluator.
Alexa+ adapter	Valid request maps; invalid IDs rejected; output schema validated; mode and errors visible.
3D house	GLB exists; room markers select correct rooms; camera avoids walls; collision/fullscreen/resize tested.
Day/night	Auto local time; manual clock; reset; midnight/timezone behavior.
UI/accessibility	Desktop/tablet/mobile no overflow; keyboard focus; icon labels; reduced motion; status not color-only.
Reports/security	Export matches run; report has versions/trace; no secrets; household scoping and privacy tested.

Definition of done

- User creates a draft, reviews structured interpretation and explicitly confirms it.
- Confirmed requirements have stable IDs and version history.
- Routine changes replay against saved requirements deterministically.
- Reports include scenario inputs, versions, ordered trace and assertion results.
- PASS/FAIL/INCONCLUSIVE are documented and tested.
- Repair is a visible diff and is unverified until rerun; related suite is rerun.
- 3D house is interactive, room navigation is safe, and UI state matches trace.
- Alexa+ route is accurately labelled and demonstrable; simulated route source is included if used.
- Existing/new tests, production build and ZIP integrity checks pass; README includes setup and limitations.

20. 90–120 second demo script

Time	Action	Narration / proof
0–10 s	Home Overview and integration badge.	“SWITCH verifies routine changes in an Alexa+ workflow. This demo is [SIMULATED / CONNECTED—state accurately].”
10–22 s	Select hallway light and saved requirement.	“The household confirmed: don't turn off the light while occupancy is true. It is versioned.”
22–35 s	Show Goodnight diff removing occupancy guard.	“A later candidate edit removed the condition; baseline remains intact.”
35–55 s	Run replay; animate occupancy=true, Goodnight fires, light OFF.	“The deterministic simulator records ordered events and resulting state.”
55–70 s	Show FAIL and trace; highlight light in house.	“This event violates the saved requirement.”
70–83 s	Show proposed fix restoring guard.	“The assistant proposes a constrained repair; the suggestion itself is not verification.”
83–103 s	Rerun target + relevant suite.	“The verifier reruns the same inputs and reports the actual result.”
103–115 s	Show report/version/reviewer decision.	“The user reviews evidence before accepting a change; modeled PASS is not a safety guarantee.”
115–120 s	Close on USP.	“A fix today should not become a failure tomorrow.”

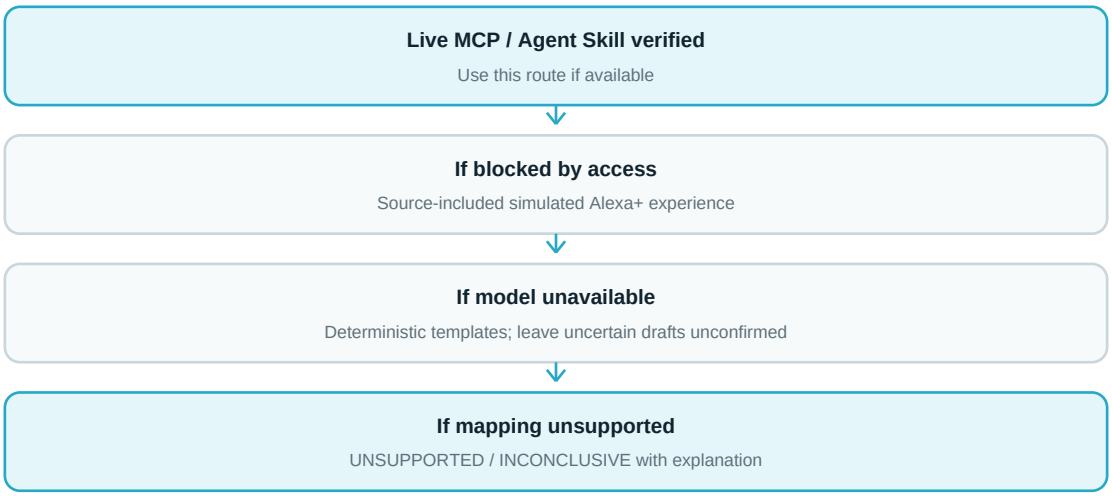
Demo checklist

- Reset demo and verify repeatability before recording.
- Show integration status truthfully; do not imply live calls when using fixtures.
- Keep credentials and private household data off-screen.
- Show failure and rerun evidence within the first 90 seconds; do not spend the demo only orbiting the house.

21. Risks, fallback and judge Q&A;

Risk / question	Response / mitigation
Alexa+ preview or credentials unavailable	Use the allowed simulated-experience route; include source and label it clearly; keep adapter boundary for later live integration.
Does SWITCH control real Alexa devices?	Answer from tested evidence only. If simulated, say so. If connected, state exactly which supported operation was demonstrated.
How is it different from Alexa testing tools?	Hypothesis is consumer correction → confirmed versioned requirement → future replay. Do not claim categorical uniqueness.
How do you know the fix is safe?	No physical safety guarantee. Show deterministic evidence for modeled requirements; human reviews/approves.
Ambiguous request?	Ask a clarifying question; keep DRAFT until confirmed.
Unknown sensor?	INCONCLUSIVE, never PASS.
Wrong AI fix?	Validate diff, show it, rerun regression suite, reject if assertions fail.
Why AI?	Interpret natural language, clarify, propose constrained repair—not determine outcomes.
What is new in this submission?	Separate pre-existing SWITCH work from changes actually made during the hackathon.

Fallback ladder



22. Research basis, references and final handoff

The original SWITCH PDF remains the source for product concept, UX direction, deterministic trust boundary, requirement lifecycle, result semantics and team workstreams. These references inform the Alexa+ adaptation; verify current docs and eligibility before implementation because preview requirements can change.

Original source preserved

- SWITCH Updated Product, UX & USP Master Document, 22 October 2026 (uploaded source PDF).
- Original core workflow: correction → interpret → clarify → confirm → version → candidate edit → select tests → replay → verify → explain → suggest → rerun → review → record.
- Original principle: build a verifiable workflow first; make it beautiful without allowing visual polish to imply unimplemented capabilities.

Amazon / Alexa+ references

- Amazon Alexa+ Automated Testing Service: https://developer.amazon.com/docs/alexaplus/smarthome/automated-testing-service.html
- Alexa+ ATS — Create test plans: https://developer.amazon.com/docs/alexaplus/smarthome/ats-create-test-plans
- Alexa+ add-ons: https://developer.amazon.com/docs/alexaplus/add-ons/home
- Alexa+ MCP toolkit quickstart: https://developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-quickstart
- Amazon Developer Hackathon track instructions: use the specific Build, Ship, Shape: Amazon Developer Hackathon page supplied by the team.

Related preliminary research from original PDF

- TAPFixer: https://arxiv.org/abs/2407.09095
- SHACR: https://arxiv.org/abs/2606.22312
- Hirz: https://github.com/BashaarJavaid/Hirz
- User-centric evaluation of smart-home conflict resolution: https://experts.illinois.edu/en/publications/a-user-centric-evaluation-of-smart-home-resolution-approaches-for/

Final product principle

Preserve SWITCH's original product and evidence-driven verification core. Make Alexa+ the interaction/integration context. Never imply live integration, physical control or guaranteed safety unless implemented, tested and evidenced.

Final handoff checklist

- Inspect project ZIP/repository and identify source of truth; create backup and record baseline tests/build.
- Agree IDs, schemas, statuses and adapter route across all three contributors.
- Implement additive changes; preserve existing tests, assets, docs and commit history.
- Run canonical Goodnight/hallway occupancy scenario end-to-end.
- Verify room tours, camera collision, fullscreen, day/night, responsive UI and error states.
- Verify live route or label simulated route; include exact setup and limitations.
- Update README, architecture/contracts docs, Alexa+ integration guide, verification report and demo script.
- Create ZIP without node_modules/secrets/cache; clean extract → install → test → build.
- Deliver ZIP, run commands, test summary, known limitations and proof of integration mode.

Recommended documentation file	Contents
README.md	Product, route chosen, setup/run/test/build, demo and integration status.
docs/ARCHITECTURE.md	Components, adapter boundary, deterministic core, data flow and trust boundaries.

Recommended documentation file	Contents
docs/CONTRACTS.md	Schemas, statuses, version semantics, API routes and trace events.
docs/ALEXA_PLUS_INTEGRATION.md	Chosen route, official requirements checked, setup/auth, supported operations, tests and limitations.
docs/VERIFICATION.md	Commands, tests, browser checks, environment, skipped checks and known issues.
.env.example	Variable names only; never real secrets.